

Módulo 1: Fundamentos y Entorno

T01: Arquitectura Web y HTTP

Carlos Rodríguez Contreras · GitHub: karrocon

 *HTTP, DNS, cliente/servidor — los cimientos de todo ataque web*

Objetivos

- Entender el flujo de una petición HTTP de extremo a extremo
- Distinguir los componentes de una arquitectura web: cliente, servidor, BD, DNS
- Leer y manipular cabeceras HTTP (herramienta: DevTools / Burp)
- Identificar qué información sensible puede viajar en una petición HTTP sin cifrar

¿Qué ocurre cuando escribes una URL?

1. Navegador → DNS resolver → IP del servidor
2. TCP 3-way handshake (SYN → SYN-ACK → ACK)
3. HTTP Request: método + path + headers + body
4. Servidor procesa → HTTP Response: status + headers + body
5. Navegador renderiza

Cada paso es una superficie de ataque potencial.

Componentes de una arquitectura web

Componente	Función
Cliente	Navegador, app móvil
DNS	Resuelve dominio → IP
CDN	Caché estática, DDoS mitigation
Reverse Proxy	Load balancing, TLS termination
Servidor App	Lógica de negocio
Base de datos	Persistencia

```
[Cliente] → [DNS] → [CDN/Proxy]
                ↓
                [App Server]
                ↓
                [Base de Datos]
```

Anatomía de una petición HTTP

```
POST /login HTTP/1.1
Host: vulnapp-owasp-01.azurewebsites.net
Content-Type: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...

{"username": "alice", "password": "password123"}
```

Parte	Qué contiene	Riesgo si viaja sin cifrar
URL	Path + query params	Visible en logs, historial
Headers	Auth tokens, cookies	Intercepción MITM
Body	Credenciales, datos PII	Lectura directa

Métodos HTTP y su semántica

Método	Acción	Idempotente	Ejemplo en VulnApp
GET	Leer	✓	GET /products
POST	Crear	✗	POST /login
PUT	Actualizar	✓	PUT /products/1
DELETE	Borrar	✓	DELETE /products/1

⚠ En VulnApp: PUT /products/:id no verifica el propietario → veremos en T08 (IDOR).

Códigos de respuesta HTTP

Rango	Significado	Ejemplos
2xx	Éxito	200 OK, 201 Created
3xx	Redirección	301 Moved, 302 Found
4xx	Error del cliente	401 Unauthorized, 403 Forbidden, 404 Not Found
5xx	Error del servidor	500 Internal Server Error

💀 Un servidor que devuelve stack traces en respuestas 500 está filtrando información interna (T13).

Headers de seguridad — vistazo rápido

Header	Protege contra
Strict-Transport-Security	Downgrade HTTP
Content-Security-Policy	XSS
X-Frame-Options	Clickjacking
X-Content-Type-Options	MIME sniffing

Los veremos en profundidad en **T13 — Headers HTTP**.

DevTools — tu primera herramienta de auditoría

1. **Network tab** — ver cada petición/respuesta en detalle
2. **Console** — ejecutar JS en el contexto de la página
3. **Application** → **Cookies** — inspeccionar flags de sesión
4. **Security tab** — verificar certificado TLS

💡 **Atajo:** `F12` o `Ctrl+Shift+I` en cualquier navegador.

DNS – ataques al sistema de resolución

Ataque	Mecanismo	Impacto
DNS Spoofing	Envenenar la caché del resolver	Redirigir tráfico a servidor falso
DNS Hijacking	Modificar registros en el registrar	Control total del dominio
Typosquatting	Registrar <code>goooogle.com</code>	Phishing, distribución de malware
DNS Tunneling	Codificar datos en queries DNS	Exfiltración desde redes restringidas

Mitigación: DNSSEC (firma criptográfica), DNS-over-HTTPS (DoH), monitorización de registros.

URL Encoding y manipulación

Carácter	→ Encoding	Uso en ataque
espacio	→ %20 o +	Bypass de filtros
/	→ %2F	Path traversal: ../%2F../%2Fetc%2Fpasswd
<	→ %3C	XSS: %3Cscript%3E
'	→ %27	SQLi: %27%20OR%201%3D1%20--
null	→ %00	Truncamiento de cadenas

⚠ Siempre decodificar **antes** de validar. Un WAF que inspecciona la URL codificada puede ser evadido con doble encoding (`%252F`).

HTTP/1.1 vs HTTP/2 vs HTTP/3

Aspecto	HTTP/1.1	HTTP/2	HTTP/3
Conexiones	Una por recurso	Multiplexado (1 TCP)	Multiplexado (QUIC/UDP)
Headers	Texto plano	Comprimidos (HPACK)	Comprimidos (QPACK)
Cifrado	Opcional (HTTPS)	Casi siempre TLS	Siempre TLS 1.3
Head-of-line blocking	Sí	Parcial	No

💡 HTTP/2 introdujo nuevos vectores de ataque: **Rapid Reset** (CVE-2023-44487) permitió DDoS masivos explotando cancelación de streams.



Lab T01: Analizando tráfico HTTP

Objetivo: interceptar y leer una petición HTTP real

Setup: `cd VulnApp && npm start` → abrir `http://localhost:3000`

1. Abre DevTools → pestaña Network
2. Inicia sesión con usuario de prueba
3. Observa las cabeceras de la petición de login
4. Identifica qué datos viajarían en claro por HTTP (sin cifrar)